
C++ Quick Start

Pierre Marchand

Sep 18, 2025

The purpose of this document is to give a good starting point to learn C++. It will focus on basic features of the language, using modern good practices. References to more advanced features may be given.

It is inspired by other tutorials focusing on “Modern C++” (at least C++14...). Here are a few references:

- Videos of Kate Gregory like [Stop teaching C](#)
- [LearnCpp.com](#)
- [C++ reference](#) and his examples.
- [Microsoft documentation](#)

How to access this document

This document is written in [Sphinx](#) and can be accessed as a [website](#) or as a [pdf document](#).

Contact

If you have any suggestions or questions, please contact me via my [personal webpage](#).

Contribute

If you have a GitHub account, you can directly suggest changes or correct typos via a pull request, either by clicking on the edit button on the top right of each page, or directly in the GitHub [repository](#) containing the source of this website. If you are not familiar with pull request, you can also open an [issue](#).

CONTENTS

1	Introduction	7
1.1	What is C++?	7
1.2	To follow along	8
1.3	Changelog	9
2	First C++ programs	11
2.1	Hello world	11
2.2	Compilation	12
2.3	Basic syntax	18
3	C++ Standard Library	23
3.1	Containers	23
3.2	Algorithms	25
3.3	Memory management	25
4	Object-oriented programming	27
4.1	Classes and objects	27
4.2	Polymorphism	32
5	Templates	37
5.1	Function templates	37
5.2	Class and templates	39
6	To go further	43
6.1	Topics	43
6.2	Other materials	43

INTRODUCTION

1.1 What is C++?

1.1.1 Short History

C++ is programming language introduced by [Bjarne Stroustrup](#) in 1985. It is initially derived from the [C programming language](#), adding multiple new features.

Warning

It is possible to write C-like code in a C++ program, but it is generally not considered as a good practice.

Since it has been introduced, C++ became standardized by the International Organization for Standardization (ISO), meaning that each new version of the language is defined by a technical document published by ISO and written by an international committee of experts.

The current versions are C++03, C++11, C++14, C++17, C++20, C++23 (the current version) and C++26 (the forthcoming version). In this document, we will mainly use features from C++17.

1.1.2 Properties of C++

- **Compiled language:** files containing C++ source code are designed to be compiled, meaning that: to be understood by a specific computer, they need to be translated to machine language for this specific computer.

Compilers are tools that execute this translation, but they can go even further and try to optimize the machine code it produces from the generic C++ code you wrote. Or they can also keep more information in the machine code to ease debugging, and give warnings when compiling.

Note

Other compiled languages are fortran and C, while Python or matlab use a [different strategy](#)

- **Statically-type language,** each variable in a program has a type known at compile type.

Note

On the contrary, Python is dynamically-type language, each variable has a type, but it does not know the type before the program is executed.

- It has two main components: the core language and the [C++ Standard Library](#). The latter is a collection of containers, algorithms and many other tools built on the former. Both are standardized by ISO.

1.2 To follow along

It is assumed that the reader has a unix-like operating system (Linux, macOS, ...). In other words, we assume the reader do not have Windows, but see [About Windows](#) from [Computer Tools](#) if you are in this case.

I also assume that the reader has basic knowledge about tools such as bash. If this is not the case, we refer to [Computer Tools](#), and in particular its section on [basic tools](#).

1.2.1 Compilers

C++ is a *compiled language*, and thus we need tools to translate C++ code to machine language. Compilers are tools that execute this translation, and thus they are necessary to have on your workstation. Examples of such tools are `g++` and `clang++`. See their documentation for installation instructions.

Note

They are very common tools, and thus, they are usually already available, or they can be easily installed, via a package manager for example.

Important

In the following examples, we will use `g++`, you can just replace this command name by `clang++` if that is the available compiler on your machine.

1.2.2 Other tools

As for most languages, several tools can be used to improve your development environment:

- Syntax colouring,
- Code Formatting,
- Static-analyser,
- Debugger,
- Code completion
- Code navigation,

1.2.3 Visual Studio Code

Most tools are available or can be integrated with [VS Code](#) using the [C/C++ extension](#). See also its [documentation](#) on using C++ in VS Code.

If you are not familiar to VS Code, you can also check out [this section](#) from [Computer Tools](#).

1.3 Changelog

All notable changes to this project will be documented in this file.

1.3.1 Unreleased

Added

FIRST C++ PROGRAMS

2.1 Hello world

In the following example, we write our first C++ function that prints “Hello world!” to the standard output of the terminal:

Listing 2.1.1: main.cpp

```
#include <iostream>
int main() {
    std::cout << "Hello world!\n";
    return 0;
}
```

The function’s name is **main**, it takes no argument, the output type is an integer, called **int** in C++, and it returns 0. Its content is delimited by the brackets, each statement is ended by `;`, and to print the string `"Hello world!\n"` to the terminal, it uses `std::cout <<`.

Important

One strength of C++ is its *standard library*, which provides common data structures and functions. We use here the `include` statement to include parts of the standard library, and in this example, the part related to inputs and outputs, which contained in particular the function `cout`.

Note

The prefix `std::` (called namespace) is the common prefix for the standard library, it is here to disambiguate the case where another included library would define functions with the same name (another `cout` function for example).

The function **main** is a special function in C++. It is the entry point of the C++ program, in other words, it is the function called when we execute the compiled code.

This is the source code of a simple *hello world* program that you can copy and paste in a file called `main.cpp`. To create an executable and run it, you can use the following commands:

```
g++ main.cpp
./a.out
```

Here is your first C++ program, and it should print “Hello world!”. In the next section, you will learn more about how to organize your C++ code and how to use it to produce a C++ program.

2.2 Compilation

C++ is a *compiled language*, thus we need *compilers* to translate C++ code to machine language and produce an executable. In this section, we will see how to build a C++ program using directly compilers, and then using more advanced tools such as *Make* and *CMake*. But first, we need to better organize our files, and learn more about the process of compilation.

2.2.1 Files and compilation

C++ Source files

Having a single file, like in our first *example*, have several disadvantages:

- each change requires recompiling everything,
- you cannot easily reuse part of your code in other program,
- it can be difficult to read for large codebase.

That is why it is good practice to divide your code into several files. It is also good to separate *definition* and *declaration* of functions and classes. It makes it easier to read the interface of your functions and classes (names, type of inputs and outputs, etc.) given by the declarations, without having all the definitions at the same place.

Important

Declarations are usually written in *header files*, by convention `.hpp`, and definitions are contained in `.cpp` files.

In the case of our simple program, we can encapsulate the line that prints “Hello world” to the standard output into a function called `print`.

- You can put the declaration of the function into a *header file*, see *Header file with multiple files: hello_world.hpp*.
- You can put the definition of the function into a different file from `main.cpp`, see *Header file with multiple files: hello_world.cpp*.
- And finally, you can just add an include statement in `main.cpp` to *Main file with multiple files: main.cpp*

In this new example, the `include` statements are used to copy/paste the *header file* so that both `.cpp` files have the declaration of the function `print`.

Listing 2.2.1: Main file with multiple files: `main.cpp`

```
#include "hello_world.hpp"
int main() {
    print();
    return 0;
}
```

Listing 2.2.2: Header file with multiple files: `hello_world.hpp`

```
1 #ifndef HELLO_WORLD_HPP
2 #define HELLO_WORLD_HPP
3
4 #include <iostream>
5 void print();
6
7 #endif
```

Listing 2.2.3: Header file with multiple files: `hello_world.cpp`

```
#include "hello_world.hpp"
void print(){
    std::cout << "Hello world!\n";
}
```

Warning

In *Header file with multiple files: `hello_world.hpp`*, you can see the lines 1,2 and 7 are special, they are called *header guards*. They are here to ensure that the header file is copied/pasted only once in `.cpp` files. One common error that can happen without these, is to have a second header file including the first header, while having a `.cpp` file including both. In that case the first header would be copied/pasted twice without the header guards.

Separate compilation

They are mainly two steps in producing an executable from source code files:

- The compiler produces object files for each C++ source code. They are usually `.o` files and contain machine code for every variables, functions and classes defined in their associated `.o` file. They also refer to functions and classes declared, but not defined in the headers.
- Then, it links the object files to produce an executable. One goal of this steps is for the object files to obtain the correct addresses to all the functions and classes compiled in other object files.

Note

More exactly, there is another step involving the preprocessor, but I suggest we focus on these two steps.

Compilation for the example from *C++ Source files* is illustrated in *Compilation process*. One advantage of the compilation process is that a modification of the definition of the function `print` in `hello_world.cpp` will not require a recompilation of `main.cpp` for example.

2.2.2 Manual compilation**Single file compilation**

To compile the file introduced in *main.cpp*, you need to use the command associated to the compiler (`g++` or `clang` for example). It will produce an executable called `a.out` by default. Then, you can try to execute it to print “Hello world!” in your terminal.

```
g++ main.cpp
./a.out
```

Note

- To change the name of the output file, you can use the `-o` flag.

```
g++ main.cpp -o My_Awesome_Executable
```

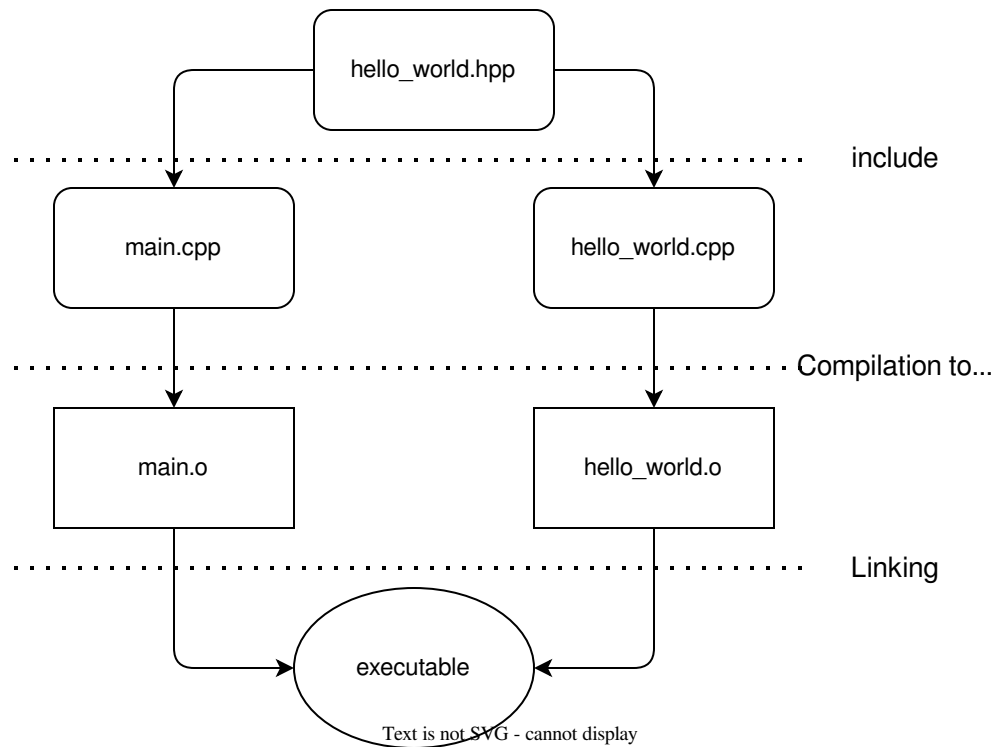


Fig. 2.2.1: Compilation process

- The two steps of the compilation can be done separately. First produce the object file using `-c` flag.

```
g++ -c main.cpp
```

It will produce the object file `main.o`. And then you can do the linking from the object file to make the executable (`a.out`).

```
g++ main.o
```

Important

Here are a few important flags when using compilers:

- `-g` generates debug information that can be used with debuggers, such as [GDB](#).
- `-Wall` enables almost all compiler warnings. It will help you understand errors in your code.

In the context of this document, we recommend using both flags.

Multiple files compilation

Once you have done that, you compile your code as follows

```
g++ -c hello_world.cpp -o hello_world.o
g++ -c main.cpp -o main.o
g++ main.o hello_world.o -o main
```

- You can recompile separately `main.cpp` and `hello_world.cpp`. So that you just need to recompile the files you modified, and redo the linking to produce the executable.
- And the files are smaller making them easier to read.

2.2.3 Make

If you have many files, multiple files compilation can be quickly cumbersome. [Make](#) is a tool made to leverage this issue. It will figure out automatically which file needs to be recompiled, and recompile just them.

Make knows which files to track, and how to recompile them using a file, called *Makefile*, which lists the different target to build, and how to build them. Using the same example as previously, you can copy and paste the code from [Simple example of makefile](#) into a file called *Makefile* alongside `main.cpp`, `hello_world.cpp` and `hello_world.hpp`.

Listing 2.2.4: Simple example of makefile

```
CC      = g++
CFLAGS  = -g -Wall
LD_FLAGS =
LIBRARY =
INCLUDE =

EXEC = main
SRCS = main.cpp hello_world.cpp
OBJS = main.o hello_world.o

all : $(EXEC)

$(EXEC) : $(OBJS)
    $(CC) $(LD_FLAGS) -o $@ $^

%.o : %.cpp
    $(CC) $(CFLAGS) -c -o $@ $<

.PHONY: clean, mrproper

clean :
    rm -rf *.o

mrproper: clean
    rm -rf $(EXEC)
```

To use Make in your terminal, you can call

- `make` to (re)compile your code, the executable will be named `main`
- `make clean` to delete all the object files,
- `make mrproper` to delete all the object files and the executable.

You can modify `hello_world.cpp`, and recall `make` to check that it will only recompile `hello_world.cpp`.

2.2.4 CMake

At least for me, it is tricky to write makefiles, and they are usually bound to one system. To make a project more platform-independent, but also to easily integrate a C++ project with your favourite source editor (see [Integration with IDEs](#) for example), another tool is often used in C++ projects: **CMake**, which will generate a `Makefile` for your system.

But before setting up CMake, let's organize better our C++ project. The standard structure for a C++ project is to put *header files* in a folder called `include`, and source files in another folder called `src`. Applying this structure to our simple example, you can then put a `CMakeLists.txt` file containing [Simple example of CMakeLists.txt](#) at the root of your project. You can find an illustration of this structure in [Simple structure for C++ project](#), where `cpp_example` is the name of the folder containing our project.

Listing 2.2.5: Simple structure for C++ project

```
cpp_example
├── CMakeLists.txt
├── include
│   └── hello_world.hpp
└── src
    ├── hello_world.cpp
    └── main.cpp
```

Listing 2.2.6: Simple example of `CMakeLists.txt`

```
1 cmake_minimum_required(VERSION 3.5)
2 project(HelloWorld)
3 add_executable(main src/main.cpp src/hello_world.cpp)
4 target_include_directories(main PRIVATE include)
5 target_compile_features(main PRIVATE cxx_std_17)
6 target_compile_options(main PRIVATE -Wall -fsanitize=address)
7 target_link_options(main PRIVATE -fsanitize=address)
```

The content of [Simple example of CMakeLists.txt](#) is relatively self-explanatory:

1. `cmake_minimum_required(VERSION 3.5)` is used to require a modern-enough version of CMake.
2. `project(HelloWorld)` defines a CMake project.
3. `add_executable(main src/main.cpp src/hello_world.cpp)` defines an executable main whose source files are `src/main.cpp` and `src/hello_world.cpp` (paths are given relatively to `CMakeLists.txt`).
4. `target_include_directories(main PRIVATE include)` specifies that to build main, headers can also be found in `include`.
5. `target_compile_features(main PRIVATE cxx_std_17)` set the C++ standard used.
6. `target_compile_options(main PRIVATE -Wall -fsanitize=address)` adds to the compilation, `-Wall` flag, which enables almost all warnings, `-fsanitize=address`, which is a tool to catch errors at runtime (out-of-bounds accesses in an array for example).
7. `target_link_options(main PRIVATE -fsanitize=address)` adds to the linking, `-fsanitize=address`.

Once you have structured your C++ project and prepared `CMakeLists.txt`, you can use [Generate Makefile](#) to generate all the necessary files and a `Makefile` into a folder `build` in `cpp_example`.

Listing 2.2.7: Generate Makefile

```
mkdir build
cd build
cmake ../
```

Note

CMake projects usually use an *out-of-source* build strategy, meaning everything build-related will be in a separate folder from the sources (here *include* and *src*). It is considered good practice:

- It separates source files, whose content is likely to be independent of the operating system and compilers, from files generated for your particular computer, compiler and compilation options. When using a version control system like [Git](#), you can ask it to ignore “build” folders, and to keep track of the source files
- You can generate several “build” folders (using different compilers, or different compilation options)

Now that the CMake project is generated, you can call `make` in `cpp_example/build` to generate the executable.

Warning

You may want to remove the flag `-fsanitize=address` from compilation and linking if you want better performance from the resulting executable. But in all other cases, it's best to keep it.

2.2.5 Integration with IDEs

By default, CMake will generate a *Unix Makefile* which can then be used to build our C++ program. But CMake can be used to generate many types of “projects” that can be used in an Integrated Development Environment (IDE): Visual Studio, XCode, CodeBlocks, etc, (see [documentation](#)).

Compilation in VS Code

If you use VS Code, you can also easily integrate your C++ program using CMake. We refer to VS Code [documentation](#) for using C++ and CMake. But to summarise:

- You need to install the extension [CMake Tools](#).
- Open the folder containing `CMakeLists.txt` with VS Code (`cpp_example` in our example).
- Use the Command Palette (`maj+ctrl/cmd+P`) and run **CMake: Select Kit** to select a compiler.
- Use the Command Palette (`maj+ctrl/cmd+P`) and run **CMake: Select Variant** to select a build type (mainly *Release*, asking the compiler to include optimizations, or *Debug*, which is needed for debugging).
- Use the Command Palette (`maj+ctrl/cmd+P`) and run **CMake: Configure** to create a `build` folder and generate the makefile, equivalent to [Generate Makefile](#).
- Use the Command Palette (`maj+ctrl/cmd+P`) and run **CMake: Build** to generate the executable.

Debugging in VS Code

If you want to run your executable with a debugger, [GDB](#) for example, and you already built your code using *Debug* mode before configuration,

- Use the Command Palette (`maj+ctrl/cmd+P`) and run **CMake: Debug** to run and debug the executable.

To help you debug your C++ program, you can use *breakpoints*:

- Click on the left of the line number, it will appear as a red dot.

When running in debug, the C++ program will stop at the breakpoints, you will then see all the current variables, and you will be able to go to the next statement or the next breakpoint. See [Debugging documentation](#) and the [Debugging C++ documentation](#) for more information.

2.3 Basic syntax

Code examples should be put in a function *main* and compiled as we have seen in [Compilation](#).

2.3.1 Variables and types

C++ requires every variable to be declared with its type. And the type of the variable cannot be changed during its lifetime. See [Example for declaring and initializing variables](#).

Listing 2.3.1: Example for declaring and initializing variables

```
// Declarations
int a;
float b;
double c;

// Initializations
int d{1};
float e{2.0};
double f{3.0};
```

It tells the compiler the size it needs to reserve in memory for every variable, and it also allows the compiler to check that every operation makes sense! If you try to compile [Example of error at compilation](#), you will get an error in your terminal:

Listing 2.3.2: Example of error at compilation

```
int a{1};
std::string str{"a string"};
std::cout << a + str;

/* Error from compiler:
error: invalid operands to binary expression ('int' and 'std::string' (aka 'basic_
↳ string<char, char_traits<char>, allocator<char>>'))
std::cout << a + str;
           ~ ^ ~~~
*/
```

Warning

`std::string` is not part of the core language, but it is defined in the *C++ Standard Library*. You need to add `#include <string>`.

2.3.2 Statements and flow control

All the usual loops are available:

- If statement: `if (condition) {statement;}` where `condition` is an expression returning a boolean. You can use the usual [condition operators](#). See *Example of if statement*
- For loop: `for (initialization; condition; increase) {statement;}`. See *Example of for loop*
- While loop: `while (expression) {statement;}`. See *Example of while loop*

Listing 2.3.3: Example of if statement

```
int x{10};
if (x>1){
    std::cout << "x is greater than 1";
}
else if (x==1){
    std::cout << "x is 1";
}
else {
    std::cout << "x is lower than 1";
}
```

Listing 2.3.4: Example of for loop

```
for (int n=0; n<10; n++) {
    std::cout << n << ", ";
}
```

Listing 2.3.5: Example of while loop

```
int n{0};
while (n<10){
    std::cout << n << ", ";
    n+=1;
}
```

2.3.3 Functions

A function's declaration is composed by its name, its return type, its parameters' type, this set is also called the function's *prototype*. As you can see in [Examples of functions](#), a function can also have no parameter or a return type `void`, meaning that it returns nothing.

Listing 2.3.6: Examples of functions

```
void print(){
    std::cout << "Hello world!" << "\n";
}

void print(int a){
    std::cout << a << "\n";
}

int add(int a, int b){
    return a+b;
}
```

Important

Functions can be overloaded. It means that you can define functions with the same name, but different parameters. See the two `print` functions in [Examples of functions](#). But you cannot overload functions with only their return type. In our examples, you cannot define `int print()`.

2.3.4 References

A reference in C++ can be seen as an alias for a variable, it is just a new name for a variable. For a variable of type `T`, the type for references to variables of such type is `T&`. See [Examples of reference](#) where we used a `string` variable, and a reference to this variable, whose type is `string &`.

Listing 2.3.7: Examples of reference

```
std::string a = "Not modified";
std::string& b=a;
b="Modified";
std::cout << a << "\n";
```

The primary use of references is related to function parameters. In [Examples of functions](#), we passed input parameters *by value*, meaning that the variables given as argument to a calling function are copied to other variables which are used in the body of the function. You can observe it in [Example of passing parameters by value](#) where we modify the argument of the function in the function body, but this does not modify the argument given in the function call. In this example, `a` is copied to `b` which is used in the function body.

Listing 2.3.8: Example of passing parameters by value

```
#include <string>
#include <iostream>

void example_by_value(std::string b){
    b = "Modified";
    std::cout << b << "\n";
}

int main(){
    std::string a = "Not modified";
    example_by_value(a);
    std::cout << a << "\n"; // prints Not Modified
}
```

To summarize, when passing by value:

- The variables used as arguments in the function call cannot be modified.
- A copy is done between the variables used as arguments in the function call, and the variables used in the function body. This copy can be costly if the variable has a large size (with a large array for example).

An alternative is to pass parameters *by reference*. In *Example of passing parameters by reference*, `b` is an alias for `a`, but the content is the same. Modifying `b` does modify `a`.

Listing 2.3.9: Example of passing parameters by reference

```
#include <string>
#include <iostream>

void example_by_reference(std::string& b){
    b = "Modified";
    std::cout << b << "\n";
}

int main(){
    std::string a = "Not modified";
    example_by_reference(a);
    std::cout << a << "\n"; // prints Modified
}
```

To summarize, when passing by reference:

- The variables used as arguments in the function call can be modified.
- There is no copy!

But, what if we want to avoid copying an argument of the function (because it is expensive), but we also want to prohibit modifying it? In this case, we can tell the compiler that the variable is a *constant reference*, meaning that this is an alias, but it cannot modify the content of the variable. A reference for a type `T` is `T&`, a constant reference for a type `T` is `const T&`. *Example of passing parameters by const reference* does not compile because we try to modify a `const int&`.

Listing 2.3.10: Example of passing parameters by const reference

```
// This example cannot be compiled! (thanks the compiler)
#include <string>
#include <iostream>
```

(continues on next page)

(continued from previous page)

```

void example_by_const_reference(const std::string& b){
    b = "Modified";
    std::cout << b << "\n";
}

int main(){
    std::string a = "Not modified";
    example_by_const_reference(a);
    std::cout << a << "\n";
}

/* Error from compiler:
error: no viable overloaded '='
b = "Modified";
~ ^ ~~~~~
*****string:905:19: note: candidate function not viable: 'this' argument has type
↳ 'const std::string' (aka 'const basic_string<char, char_traits<char>, allocator
↳ <char>>'), but method is not marked const
basic_string& operator=(const basic_string& __str);
*/

```

Note

If you use an IDE with a static analysis tool, you do not even need to compile to see that there is an issue in *Example of passing parameters by const reference*. The IDE should tell you that `b = "Modified"` is not possible. But if you try to compile, the compiler will show an error.

2.3.5 Scope and RAII

Each identifier (variables, functions, ...) defined in a C++ program has a *scope* (delimited by a pair of curly brackets `{ . . }`), which is a region of the program where it can be used. In particular for variables, a scope defines its lifetime. Objects are destructed in reverse order of initialization when exiting their scope.

Note

You can also define global variables, outside any pair of curly brackets, but this is usually considered bad practice.

One of the main feature of C++, one of the concept that makes C++ more than just C with classes is how you can handle resources (for example memory, files). In C, you need to allocate a resource, so you can use it, and then you need to delete the object to free its memory. In C++, resources can be handled automatically by binding their life cycle to the lifetime of an object.

Important

This feature is called **RAII** for *Resource Acquisition Is Initialization*, but one important point is that the resource is released automatically when the associated object is destructed.

Examples will be given in *Pointers* for managing automatically memory allocation.

C++ STANDARD LIBRARY

As we have already quickly seen, one strength of C++ is its [C++ Standard Library](#) which contains strings, ([Strings library](#)), common data structures ([Container library](#)), algorithms ([Algorithms library](#)), and common mathematical functions and types, such as complex number, random generators ([Numerics library](#)).

We review a few features available, but feel free to take a look by yourself at the documentation to see the many tools already available.

3.1 Containers

3.1.1 Arrays

One of the most common containers is an array, meaning values of a given type contiguously stored in memory. In C++, there are two types of arrays:

- Static arrays, whose size is known at compile type, which implies that they have a fixed size.
- Dynamic arrays, whose size can vary during execution.

Both are available in the C++ standard library, with respectively `std::array` and `std::vector`. To interact with objects of these types, you can use *methods*, functions that belong to the definition of these types. To call a method called `call_method`, which takes an integer as argument, on an object called `my_object`, you need to write `my_object.call_method(3)`. In [Example of arrays](#), you can see how to use vectors and arrays from STL, and some of their methods. See the section “Member functions” in the documentation of [std::vector](#) and [std::array](#)

Listing 3.1.1: Example of arrays

```
#include <iostream>
#include <vector>
#include <array>

int main()
{
    std::vector<int> zero_array_of_size_10(10,0);
    std::vector<int> dynamic_array{1, 2, 3, 4};
    std::array<int, 3> static_array{1, 2, 3};

    // Sizes
    std::cout << dynamic_array.size() << " " << static_array.size() << "\n";

    // Access element
    std::cout << dynamic_array[2] << " " << static_array[2] << "\n";
}
```

(continues on next page)

(continued from previous page)

```
// Modify vector and accessing last element
dynamic_array.push_back(5);
std::cout << dynamic_array.size() << " " << dynamic_array.back() << "\n";

return 0;
}
```

Important

As you can see in *Example of arrays*, `std::vector` and `std::array` are followed respectively by `<int>` and `<int, 3>`. They are *template arguments*, i.e., arguments parameters that indicate at *compile time* information about the actual type used. In the case of `std::vector`, it means that it is a contiguous array of `int`, while for `std::array`, it means that it is a static array of `int` whose size is 3.

To loop over the elements of a vector or an array, you could use the for loop we saw in *Statements and flow control* accessing elements and using the `size` method, but a convenient alternative is to use *range-based for loops*. The advantage of this approach is that you are guaranteed to only loop on the elements of the vector, which is not the case of the traditional for loop where segmentation fault errors can occur if the loop goes too far.

Listing 3.1.2: Example of range-based loops

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> dynamic_array{1, 2, 3, 4};
    for (int i : dynamic_array) // access by value
    {
        std::cout << i << " ";
        i = i + 1;
    }
    std::cout << "\n";

    for (int &i : dynamic_array) // access by reference
    {
        std::cout << i << " ";
        i = i + 1;
    }
    std::cout << "\n";

    for (const int &i : dynamic_array) // access by const reference
    {
        std::cout << i << " ";
    }
    std::cout << "\n";

    return 0;
}
```

3.1.2 Other types of containers

As you may know, there are many types of data containers, which may have different operations available, with different complexities. Here are a few examples (see also [documentation](#)):

- Sequential containers: `std::array`, `std::vector`, linked list as `std::list`, ...
- Associative containers: `std::set`, dictionary or the are called in C++ `std::map`, ...
- Container adaptors: `std::stack`, `std::queue`, ...

Choosing the right type of container depends on the type of operations you need, and their cost. In the documentation, the complexity is reminded. For example, [access to an element](#) of a `std::vector` via an index is constant, while access to an element of a `std::map` via its key is logarithmic in the size of the container.

3.2 Algorithms

Many standard algorithms are already implemented in the STL, and they can be easily used with standard [Arrays](#), and even *Other types of containers* if relevant. You can find more about them in the documentation of the [algorithm library](#).

Listing 3.2.1: Example of STL algorithms

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> dynamic_array{1, 4, 3, 2, 6, 5, 2};
    std::cout << std::count(dynamic_array.cbegin(), dynamic_array.cend(), 2) << "\n";
    ↪ // count the number of element 2
    std::sort(dynamic_array.begin() + 2, dynamic_array.end());
    ↪ // sort starting from the third element
    std::swap(dynamic_array.front(), dynamic_array.back());
    ↪ // swap the first element with the last element

    for (const int &i : dynamic_array) // access by const reference
    {
        std::cout << i << " ";
    }
    std::cout << "\n";

    return 0;
}
```

3.3 Memory management

3.3.1 Pointers

A pointer is a variable that stores the address in memory of a variable. For example, in *Pointer variable*, `pointer_to_a` is a variable containing the address of `a`, and to get the address from `a` we used `&`. If you try to display it with `std::cout`, it will print its address in memory. You can directly use methods from the pointed variable's type using `->`, or you can access the underlying variable by “dereferencing” the pointer using `*`.

Listing 3.3.1: Pointer variable

```
std::vector<int> a {0,1,2,3};
std::vector<int>* pointer_to_a = &a;
std::cout << pointer_to_a << " " << pointer_to_a->size() << " " << (*pointer_to_a)[0] << "\n";
```

Pointers and *References* are similar in the sense that they are both mechanisms to access an underlying variable. But there are key differences that we highlight in *References vs pointers*.

Table 3.3.1: References vs pointers

Pointers	References
Can be assigned to <code>nullptr</code>	Cannot be null
Can be re-assigned	Cannot be re-assigned

3.3.2 Smart pointers

Common issues with pointers appear when a pointer “owns” the variable it points to, in other words, when it manages the lifetime of the underlying variable. With “raw” pointers as in the *previous* section, you need to allocate manually, give its address to the pointer, and free the memory manually before the end of the program. Typical errors are

- forgetting to free the memory, *memory leaks*,
- freeing the memory too early, so that the pointer points to nothing, *dangling pointers*.

To avoid these issues, the idea is to use *RAII*. We tie the lifetime of the underlying variable to the lifetime of a type representing a pointer which owns its data. The latter is implemented in the C++ standard library as `std::unique_ptr`.

To create a `std::unique_ptr<T>` to an object of type `T`, one has to use `make_unique<T>` whose input parameters are the ones used to construct directly an object `T`.

Listing 3.3.2: Example with `std::unique_ptr`

```
// you need to use #include <memory>
std::unique_ptr<std::vector<int>>> owning_ptr_to_a_vector;
owning_ptr_to_a_vector = std::make_unique<std::vector<int>>>(10);
std::cout << owning_ptr_to_a_vector->size() << "\n"; // its size is 10
```

Note

In C, one uses `malloc` and `free` to allocate and free memory. In C++, it is also possible with `new` and `delete`, but it is discouraged since `std::unique_ptr` does it for you automatically. But it is not rare to see manual memory management in legacy C++ code, or specialized library.

OBJECT-ORIENTED PROGRAMMING

Object-oriented programming (OOP) is a programming paradigm where data will be encapsulated in “objects”, which also contain functions to interact with the data. These functions are called “methods” and we already saw examples of such functions when using `std::vector` and `std::array` in [Arrays](#).

Object-oriented programming allows defining a specific interface to interact with a set of data.

4.1 Classes and objects

We saw in [Variables and types](#) that the language itself provides common types like numerical types (`int`, `float`, etc.), and in [C++ Standard Library](#), we gave a few examples of predefined types such as `std::vector`. One can also define its own custom types for its particular needs. In C++, a type is defined with the keywords `class` or `struct`, and we call *objects* particular instances of a type.

4.1.1 Data members

Data members are the encapsulated data in the class. In [Custom object with data members](#), the custom class `Point` encapsulates two `double` in lines 3 and 4, its x and y coordinates. They are in the `private` section of the class, meaning we cannot directly access them. If instead we had put the data member in a `public` section, then they would be directly accessible using `.` on the object once created.

Listing 4.1.1: Custom object with data members

```
1 // in point.hpp
2 class Point {
3     private:
4         double m_x;
5         double m_y;
6
7 };
```

Note

Several conventions exist on the naming of data members, we use here the prefix `m_` for member.

Note

The difference between `class` and `struct` is that by default, what is declared inside is private for the former, and public for the latter.

Important

We still use separation compilation as seen in *Separate compilation*, and we comment out in which file the lines of code are contained.

4.1.2 Constructors

The class can have constructors initializing the encapsulated data, as in line 10 of *First example of custom class*. It can actually have several constructors, we could have added another constructor taking only one `double` which initializes both `m_x` and `m_y` for example.

Listing 4.1.2: First example of custom class

```

1 // in point.hpp
2 class Point {
3     private:
4         double m_x;
5         double m_y;
6     public:
7         Point(double x, double y) : m_x(x), m_y(y) {}
8
9 };

```

Another important point is the difference between using the constructor of the data members vs using their assignment operator. In *Constructor vs copy for initializing data members*, both approaches are possible, but the code on the left uses directly the constructor of `double` to initialize the values of `m_x` and `m_y`, while in code on the right, the two values are initialized without a given value, and then we assign them their values. It is the same difference as in `double a(3);` vs `double a; a = 3;`, which can matter if instead of `double` we have an object whose constructor is costly, or which does not have an assignment operator `operator=`.

Table 4.1.1: Constructor vs copy for initializing data members

```

Point(double x, double y) : m_x(x), m_
    ↪ y(y) {
}

```

```

Point(double x, double y) {
    m_x=x; m_y=y;
}

```

4.1.3 Member functions

Once we have constructed our object, we want to interact with it. Thus, we can add *member functions*, also called *methods*, that interact with the encapsulated data. In *Custom class with member functions*, we add three functions declared in the class `Point`. Note that they can access to the encapsulated data, and they have the `const` keyword, which tells the compiler that these functions do not modify the encapsulated data.

Listing 4.1.3: Custom class with member functions

```

1 // in point.hpp
2 #include <iostream>
3 class Point {
4     private:
5         double m_x;
6         double m_y;
7     public:
8         Point(double x, double y): m_x(x), m_y(y) {}
9         double x() const {return m_x;}
10        double y() const {return m_y;}
11        double squared_norm() const {return m_x * m_x + m_y * m_y;}
12
13 };
14
15 // in main.cpp
16 #include "point.hpp"
17 int main(){
18     Point my_point(1,2);
19     std::cout << my_point.x() << my_point.y() << " " << my_point.squared_norm() << "\n";
20 }

```

Note

As for *Functions*, member functions can be overloaded, meaning we can define member functions with the same name but different input parameters.

4.1.4 Operator overloading

Operators are symbols that performs operations, such as +, −, etc. They can be defined for custom classes. We will give a few examples and we refer to the [documentation](#) for an exhaustive list.

Depending on the particular symbol, operators can be member functions, typical example for `Point` is given in lines 15-19 of *Custom objects with member functions and operators*.

Important

Member functions are always applied to an object of the class, so member functions have always an implicit argument which is the particular object on which they are applied. This implicit argument can be accessed using the keyword `this`, which is a pointer to the object.

Note

*`this` would be similar to `self` in python.

Other symbols can be defined as functions, see lines 21-26 from *Custom objects with member functions and operators*.

Listing 4.1.4: Custom objects with member functions and operators

```

1 // in point.hpp
2 #include <iostream>
3
4 class Point
5 {
6 private:
7     double m_x;
8     double m_y;
9
10 public:
11     Point(double x, double y) : m_x(x), m_y(y) {}
12     double x() const { return m_x; }
13     double y() const { return m_y; }
14     double squared_norm() const { return m_x * m_x + m_y * m_y; }
15
16     Point &operator*=(double t) {
17         m_x *= t;
18         m_y *= t;
19         return *this;
20     }
21 };
22
23 Point operator+(const Point &u, const Point &v);
24 std::ostream &operator<<(std::ostream &out, const Point &v);
25
26 // in point.cpp
27 #include "point.hpp"
28 Point operator+(const Point &u, const Point &v) {
29     return Point(u.x() + v.x(), u.y() + v.y());
30 }
31 std::ostream &operator<<(std::ostream &out, const Point &v) {
32     return out << v.x() << ' ' << v.y();
33 }
34
35 // in main.cpp
36 #include "point.hpp"
37 int main() {
38     Point my_point(1,2);
39     std::cout << my_point + my_point << " " << my_point.squared_norm() << "\n";
40 }

```

4.1.5 Life cycle of a custom object

Some member functions and operators are special because they are related to the lifetime of objects:

- Constructors and in particular the **default constructor** which is used with `Point A;`
- **Copy constructors**, its declaration is `Point (const Point&)`, and it is used with `Point B(A);`
- **Copy assignment**, its declaration is `Point& operator=(Point)`, and it is used with `Point B=A;`
- **Destructors**, `~Point()` which is automatically called when exiting the scope where the object is defined.

They can be difficult to implement, that is why they are implicitly defined in some cases. But the actual rules to understand when they are implicitly defined and when they are not are quite subtle. To keep things simple:

Try to always use data members from the core language or the C++ standard library (or objects that are composed of them).

- You can rely on their own constructors/copy constructor/copy assignment/destructor to define the ones of your class, and they will be implicitly defined as long as you do not define one of them manually. This is called the [rule of zero](#)
- The only exception being the default constructor that is not implicitly defined as soon as you define another constructor. To automatically define it using the default constructors of each data members, you can add in the class definition `Point() = default;`.

Note

A typical way to **break** this rule is to have a pointer as a data member and to manually manage the memory with `new` and `delete`. It is one reason why it is discouraged.

Important

Not all types are copyable, for example, `std::unique_ptr` from *Smart pointers* is not. Thus, having a `std::unique_ptr` as a data member makes the class non-copyable.

Note

Move semantic is out of the scope, but the remarks here extend to [Move constructor](#) and [Move assignment](#).

4.1.6 Separate definition and declaration

So far, we have put the definitions and declarations of the member functions in a *header* file, but as we have seen before, it is generally good practice to separate definition from declaration. We can still do it with function members, the only difference being that since they are defined in the *namespace* of our custom class, we need to put the prefix `Point::` when defining them outside the class, see lines 223-226 in *Custom objects with only declaration*.

Listing 4.1.5: Custom objects with only declaration

```

1  // in point.hpp
2  #include <iostream>
3
4  class Point
5  {
6  private:
7      double m_x;
8      double m_y;
9
10 public:
11     Point(double x, double y) : m_x(x), m_y(y) {}
12     double x() const;
13     double y() const;
14     double squared_norm() const;
15     Point &operator*=(double t);
16 };
17
```

(continues on next page)

(continued from previous page)

```

18 Point operator+(const Point &u, const Point &v);
19 std::ostream &operator<<(std::ostream &out, const Point &v);
20
21 // in point.cpp
22 #include "point.hpp"
23 double Point::x() const { return m_x; }
24 double Point::y() const { return m_y; }
25 double Point::squared_norm() const { return m_x * m_x + m_y * m_y; }
26 Point &Point::operator*=(double t) {
27     m_x *= t;
28     m_y *= t;
29     return *this;
30 }
31
32 Point operator+(const Point &u, const Point &v){
33     return Point(u.x() + v.x(), u.y() + v.y());
34 }
35 std::ostream &operator<<(std::ostream &out, const Point &v){
36     return out << v.x() << ' ' << v.y();
37 }
38
39 // in main.cpp
40 #include "point.hpp"
41 int main(){
42     Point my_point(1,2);
43     std::cout << my_point + my_point << " " << my_point.squared_norm() << "\n";
44 }

```

4.2 Polymorphism

In programming languages, **polymorphism** is a concept referring to entities (variable, function or object) having *multiple forms*, meaning multiple definitions. We already saw two examples previously:

- *function overloading*: a function can have multiple definitions depending on its number of parameters and their types,
- *Operator overloading*: the same operator (for example +) can have multiple meaning for different types.

In these two examples, the specific form taken by the function or the operator is chosen at compile-time. In the following, we will present *runtime polymorphism*, which is related to inheritance and overriding function members.

4.2.1 Inheritance

Like in many OOP languages, it is possible to define a custom class that inherits data members and methods from another class. We call the latter *base class*, and the former *derived class*. Given a custom class `Character` as in lines 3-14 of *Inheritance with function overriding*, one can implement a derived class as in line 15 of *Inheritance with function overriding* adding `: public Character` to the name of the class.

Note

public inheritance means that the derived class cannot access private member of the base class, but it can access public members. There is also a new section called `protected` in the base class, members in these sections are

only accessible by derived class. In particular, Farmer can access `m_name` and `m_life_points`.

Listing 4.2.1: Inheritance with function overriding

```

1 #include <string>
2 #include <iostream>
3 class Character {
4     protected:
5         std::string m_name;
6         unsigned int m_life_points;
7     public:
8         Character(std::string name, unsigned int life_points): m_name(name), m_life_
↳points(life_points) {}
9         void who_am_i(){
10             std::cout << "My name is "<< m_name;
11             std::cout << " and I am a character with "<< m_life_points<< " life_
↳points.\n";
12         }
13 };
14 class Farmer : public Character {
15     private:
16         std::string m_sound;
17     public:
18         Farmer(std::string name, unsigned int life_points): Character(name,life_
↳points) {}
19         void who_am_i(){
20             std::cout << "My name is "<< m_name;
21             std::cout << " and I am a farmer with "<< m_life_points<< " life points.\n
↳";
22         }
23 };

```

In this example,

- we factorized the common parts of a character: its name and number of life points, and we specialized it for a specific case, a farmer. The advantage now is that we could add other derived classes, and benefit from the already implemented constructor of `Character`,
- we can also override member function from the base class in a derived class, which is done for `who_am_i`. In *Use derived class* line 3, we show how `who_am_i` for a `Farmer` instance uses the one defined in `Farmer`.

Listing 4.2.2: Use derived class

```

1 int main() {
2     Farmer farmer("John", 10);
3     farmer.who_am_i(); // My name is John and I am a farmer with 10 life points.
4     Character &farmer_as_character = farmer;
5     farmer_as_character.who_am_i(); // My name is John and I am a character with 10_
↳life points.
6 }

```

Important

Using a base class reference or pointer to an instance of a derived class is correct, but overridden member functions will use the base implementation. For example in *Use derived class* line 5, `who_am_i` uses its definition from the

Character class. To avoid this behavior, see [Virtual member functions](#).

4.2.2 Virtual member functions

We saw in [Use derived class](#) that the derived implementation of the overridden member function was not chosen during the execution of the program when using a reference. To make it possible, we need to define the member function as virtual.

Listing 4.2.3: Inheritance with virtual member function

```

1 // in character.hpp
2 #include <string>
3 #include <iostream>
4 class Character {
5     protected:
6         std::string m_name;
7         unsigned int m_life_points;
8     public:
9         Character(std::string name, unsigned int life_points): m_name(name), m_life_
↳ points(life_points) {}
10        virtual void who_am_i() {
11            std::cout << "My name is " << m_name;
12            std::cout << " and I am a character with " << m_life_points << " life_
↳ points.\n";
13        }
14    };
15    class Farmer : public Character {
16        private:
17            std::string m_sound;
18        public:
19            Farmer(std::string name, unsigned int life_points): Character(name, life_
↳ points) {}
20            void who_am_i() {
21                std::cout << "My name is " << m_name;
22                std::cout << " and I am a farmer with " << m_life_points << " life points.\n
↳ ";
23            }
24    };
25
26    int main() {
27        Farmer farmer("John", 10);
28        farmer.who_am_i(); // My name is John and I am a farmer with 10 life points.
29        Character &farmer_as_character = farmer;
30        farmer_as_character.who_am_i(); // My name is John and I am a farmer with 10 life_
↳ points.
31    }

```

Runtime polymorphism allows defining more generic functions. In our example, we could define a function taking a reference or a pointer to the base class, and it would accept any derived class from the base class (present and future!). Thus, we can write code using the interface defined by the base class.

Listing 4.2.4: Function taking base class as input

```
void who_is(Character& character){character.who_am_i();}
```

Note

It is called runtime polymorphism because, even if there is no information at compile-time, the correct derived implementation is invoked during the execution of the program.

4.2.3 Pure virtual member functions

It may be better to actually not be able to construct an instance of a base class, which may not mean anything without a proper definition. In our example, it may not mean a lot to have an instance of `Character`. To prevent this, one can use *pure virtual methods* adding `=0` as in line 10 of *Inheritance with pure virtual member function*. Then, the base class `Character` cannot be instantiated, the class is said to be *abstract*, and derived classes are required to implement the pure virtual member function to be instantiable.

Listing 4.2.5: Inheritance with pure virtual member function

```
1 // in character.hpp
2 #include <string>
3 #include <iostream>
4 class Character {
5     protected:
6         std::string m_name;
7         unsigned int m_life_points;
8     public:
9         Character(std::string name, unsigned int life_points): m_name(name), m_life_
↳ points(life_points) {}
10         virtual void who_am_i() =0;
11 };
12 class Farmer : public Character {
13     private:
14         std::string m_sound;
15     public:
16         Farmer(std::string name, unsigned int life_points): Character(name, life_
↳ points) {}
17         void who_am_i() override{
18             std::cout << "My name is "<< m_name;
19             std::cout << " and I am a farmer with "<< m_life_points<< " life points.\n
↳ ";
20         }
21 };
22
23 int main(){
24     Farmer farmer("John", 10);
25     farmer.who_am_i(); // My name is John and I am a farmer with 10 life points.
26     Character &farmer_as_character = farmer;
27     farmer_as_character.who_am_i(); // My name is John and I am a farmer with 10 life_
↳ points.
28 }
```

Note

We added the keyword `override` in the definition of `who_am_i` in the derived class. It indicates to the compiler that the function is meant to override a virtual function from a base class, and compilation will fail if it does not.

It helps to detect common errors where the prototype of the function has a typo (missing argument, missing `const` keyword, etc.) and thus is not overriding the function from the base class but just adds a new function member using overload (using *Member functions*).

Note

One usage of base classes with pure virtual member function is to define interfaces. It helps keeping particular implementations of interfaces (such as `Farmer` in our example) separate from other parts of the code that only depend on the interface.

Note

Another example more in the context of scientific computing:

- Define an interface (with an abstract class) for an operator with a matrix-vector product (modelled with a pure virtual function).
- Different implementations (derived classes from the abstract class) can be defined: dense matrix, sparse matrix, compressed matrix, etc.
- Then, iterative linear solvers such as GMRES or CG can be defined only depending on the interface (abstract class) without depending directly on the particular matrix implementations.

TEMPLATES

Templates are a C++ mechanism that allows writing more generic code by parametrizing types. We already encountered this when using `std::vector` in *Arrays*. Indeed, we saw that `std::vector` was followed by `<int>`, specifying the actual type of the stored elements. Types and functions from the *C++ Standard Library* are usually defined for generic types. For example, one could use `std::vector<double>`, `std::vector<std::string>` or even user-defined *Classes and objects* such as `std::vector<Point>`.

5.1 Function templates

5.1.1 Definition

To parametrize the types used in a function, we define *template types* as in line 1 from *Function template definition*. Function template can have both template parameters and non-template parameters like `multiply_by_N`.

Listing 5.1.1: Function template definition

```
1 template<typename T>
2 multiply_by_ten(T a){
3     return a * 10;
4 }
5
6 template <typename T>
7 multiply_by_N(T a, int N){
8     return a * N;
9 }
```

Important

A function template cannot be compiled directly as is. Since it is defined for generic/abstract types, it does not represent an actual definition of a function that can be compiled into language machine. Thus, definition **and** declaration should be in header files.

Note

Function templates are really “templates” for defining actual functions at compilation time. Thus, they allow generating code, and that is why templates in C++ are referred as a tool for metaprogramming.

5.1.2 Use

To use a function generated from a function template, we can do as we saw with `std::vector` in *Arrays*, specifying the actual type. Or, we can rely on the compiler *template argument deduction*, where it uses the types of the arguments to deduce the template argument. Both are illustrated in *Function template use*.

Listing 5.1.2: Function template use

```

1  template<typename T>
2  T multiply_by_ten(T a){
3      return a * 10;
4  }
5
6  int main(){
7      std::cout << multiply_by_ten<int>(2)<<"\n"; // 20
8      std::cout << multiply_by_ten(2)<<"\n"; // 20
9      std::cout << multiply_by_N(2,2)<<"\n"; // 4
10     // std::cout << multiply_by_ten("2")<<"\n"; // does not compile
11 }

```

Note

Function template may not compile if the substitution of the template type does not lead to meaningful operations. In *Function template use*, `multiply_by_ten("2")` cannot compile because `operator*` is not defined in `"2"*10`.

5.1.3 Specialization

The definition of a template function can be modified for a particular value of template type. This is called *template specialization*.

Listing 5.1.3: Function template specialization

```

1  #include <iostream>
2
3  template <typename T>
4  void print(T a){
5      std::cout << a << "\n";
6  }
7
8  template <>
9  void print(std::pair<int, int> a){
10     std::cout << a.first << " " << a.second << "\n";
11 }
12
13 int main(){
14     print(2);
15     print(std::pair<int,int>(2,3)); // would not compile without specialization.
16 }

```

Note

If a function template has several template types, specialization will work only if all template types are specialized.

5.2 Class and templates

5.2.1 Class template definition

Very similarly to *Function templates*, class template can be defined as *Class template definition*.

Listing 5.2.1: Class template definition

```

1  #include <array>
2  template <typename T>
3  class vec3
4  {
5  private:
6      std::array<T, 3> m_data;
7
8  public:
9      vec3(T x, T y, T z) : m_data{x, y, z} {}
10     T norm() { return std::sqrt(m_data[0] * m_data[0] + m_data[1] * m_data[1] + m_
11     ↪data[2] * m_data[2]); }
12 };

```

5.2.2 Use of class template

A class template can always be used by explicitly specifying the type. But unlike *function templates*, *class template argument deduction* was only available starting from C++17.

Listing 5.2.2: Class template use

```

1  #include <array>
2  template <typename T>
3  class vec3
4  {
5  private:
6      std::array<T, 3> m_data;
7
8  public:
9      vec3(T x, T y, T z) : m_data{x, y, z} {}
10     T norm() { return std::sqrt(m_data[0] * m_data[0] + m_data[1] * m_data[1] + m_
11     ↪data[2] * m_data[2]); }
12 };
13
14 int main() {
15     vec3<double> my_vec3(1.5, 2.5, 3.5);
16     vec3 my_vec3(1.5, 2.5, 3.5); // only work for C++17 and newer
17 }

```

5.2.3 Member templates

A class (template or not) can also contain template functions.

Listing 5.2.3: Member templates

```

1  #include <iostream>
2  #include <fstream>
3
4  template <typename P>
5  struct Printer
6  {
7  private:
8      P &m_output;
9
10 public:
11     Printer(P &output) : m_output(output) {}
12     template <typename T>
13     void operator()(const T &a) { m_output << a << "\n"; }
14 };
15
16 int main()
17 {
18
19     Printer terminal_printer{std::cout};
20     std::ofstream file("outputfile");
21     Printer file_printer{file};
22     terminal_printer(3);
23     terminal_printer("hello");
24     file_printer("test");
25 }
```

5.2.4 Specialization

Class templates and member templates can also be specialized like *function templates*.

Listing 5.2.4: Member templates

```

1  #include <iostream>
2  #include <fstream>
3
4  template <typename P>
5  struct Printer
6  {
7  private:
8      P &m_output;
9
10 public:
11     Printer(P &output) : m_output(output) {}
12     template <typename T>
13     void operator()(const T &a) { m_output << a << "\n"; }
14     template <>
15     void operator()(const double &a) { m_output << std::scientific << a << "\n"; }
16 };
17
18 int main()
```

(continues on next page)

(continued from previous page)

```
19 {  
20  
21     Printer terminal_printer{std::cout};  
22     std::ofstream file("outputfile");  
23     Printer file_printer{file};  
24     terminal_printer(3);  
25     terminal_printer("hello");  
26     file_printer("test");  
27 }
```


TO GO FURTHER

6.1 Topics

- Concepts
- Move semantic
- constexpr
- lambdas
- enum

6.2 Other materials

- **Video**
 - **Tutorials**
 - * [C++ Weekly With Jason Turner](#)
 - * [The Cherno](#) (his C++ playlist for example)
 - **Conferences**
 - * [CppCon](#)
 - * [Meeting Cpp](#)
 - * [CppNow](#)
- **Blogs**
 - [C++ Stories](#)
 - [Modernes C++](#)
- **Podcasts**
 - [UCL for Code in Research](#)
 - [CppCast](#)